Boolean Parenthesization Problem

Problem Statement

Given a boolean expression with following symbols.

```
Symbols
    'T' ---> true
    'F' ---> false
```

```
Operators
    &   ---> boolean AND
    |   ---> boolean OR
    ^   ---> boolean XOR
```

Count the number of ways we can parenthesize the expression so that the value of expression evaluates to true.

Let the input be in form of two arrays one contains the symbols (T and F) in order and other contains operators (&, | and ^}

Examples

```
Input: symbol[]    = {T, F, T}
       operator[]  = {^, &}
Output: 2
The given expression is "T ^ F & T", it evaluates true
in two ways "((T ^ F) & T)" and "(T ^ (F & T))"
```

```
Input: symbol[]    = {T, F, F}
       operator[]  = {^, |}
Output: 2
The given expression is "T ^ F | F", it evaluates true
in two ways "( (T ^ F) | F )" and "( T ^ (F | F) )".
```

```
Input: symbol[]    = {T, T, F, T}
       operator[]  = {|, &, ^}
Output: 4
The given expression is "T | T & F ^ T", it evaluates true
in 4 ways ((T|T)&(F^T)), (T|(T&(F^T))), (((T|T)&F)^T)
and (T|((T&F)^T)).
```

Analysis

Let

- T(i,j) is number of ways expression from i to j can be parenthesized so that it evaluates to True
- F(i,j) is number of ways expression evaluates to False.
- Total(i,j) = T(i,j) + F(i,j) is the total number of ways an expression can be parenthesized irrespective of out being True or False

What will be the most trivial Boolean expression?

Of course, an expression with only one Boolean value T or Boolean value F.

How many ways can this expression be parenthesized so that expression evaluates to True ?

Apparently, there is only one way. For Boolean value T, there is one way, (T); whereas for F, there's no way we can parenthesize to evaluate it to True. An expression can evaluate to either True or False value. Hence the base case is when i==j:

```
T(i,i) = 1 if operand is T
         0 if operand is F

F(i,i) = 0 if operand is T
         1 if operand is F
```

Analysis

Q: How to calculate T(i, j) for expression with more than one values and operators between them?

Ans:

This is something familiar to matrix chain multiplication problem.

We will put parenthesis at all possible position and count how many ways these two resultant expressions hold True. Once we have count for each expression, we can combine count based on operator between split expression.

For expression from index i to index j, find k such that $i<k<j$, and find number of ways expressions from i to k and k+1 to j evaluates to True.

We have to solution for T(1,N).

For & (and) operator

Q: When can Expression(i,j) be True if expression is of form Expression(i, k) & Expression(k+1, j)?

A:

Only if Expression(i,k) and Expression(k+1,j) are both True.

Hence, for any k, expression can be True in T(i,k) * T(k+1, j) where T(i,k) is number of ways Expression(i,k) is True and T(k+1, j) is number of ways Expression(j+1, j) is True.

T(i,j) = Summation(T(i,k) x T(k+1,j)) where i<k<j

Total(i,j) = Total(i,k) x Total(k+1,j)

For & (and) operator

Q: When can Expression(i,j) be False if expression is of form Expression(i, k) & Expression(k+1, j)?

A: When one of the two expressions evaluate to false.

```
  Total(i,j) = T(i,j) + F(i,j)
  Total(i,j) = Total(i,k) x Total(k+1,j)


  Total(i,j) = T(i,j) + F(i,j)
=>F(i,j) = Total(i,j) - T(i,j)
=>F(i,j) = Summation(Total(i,j) - T(i,k) * T(k+1,j)) where i<k<j
=>F(i,j) = Summation(Total(i,k) x Total(k+1,j) - T(i,k) * T(k+1,j)) where i<k<j
```

For | (or) operator

Q: When can Expression(i,j) be True if expression is of form Expression(i, k) | Expression(k+1, j)?

A:

In case, operator is OR, then, whole expression is True if any one of the expressions is True.

So, How many ways both Exp(i,k) and Exp(k+1, j) be False.

$$F(i,j) = \text{Summation } (F(i,k) * F(k+1,j)) \text{ for all } i<k<j$$

Overall expression is True when both sub-expressions are not False.

Hence:

$$T(i,j) = \text{Summation } (\text{Total}(i,j) - F(i,k) * F(k+1,j)) \text{ where } i<k<j$$

For ^ (xor) operator

Q: When can Expression(i,j) be True if expression is of form Expression(i, k) ^ Expression(k+1, j)?

A:

In case, operator is XOR, then, whole expression is True if

1. Expression(i,k) is True and Expression(k+1, j) is False or
2. Expression(i,k) is False and Expression(k+1, j) is True

Hence:

$$T(i,j) = \text{Summation}\ (\ T(i,k)*F(k+1,j)\ +\ F(i,k)*T(k+1,j))\ \text{where}\ i<k<j$$

$$T(i,j) = \sum_{k=i}^{j-1} \begin{cases} T(i,k) * T(k+1,j) & \text{If operator[k] is '\&'} \\ Total(i,k) * Total(k+1,j) - F(i,k) * F(k+1,j) & \text{If operator[k] is '|'} \\ T(i,k) * F(k+1,j) + F(i,k) * T(k+1) & \text{If operator[k] is '^'} \end{cases}$$

$$\mathrm{Total(i, j) = T(i, j) + F(i, j)}$$

$$F(i,j) = \sum_{k=i}^{j-1} \begin{cases} Total(i,k) * Total(k+1,j) - T(i,k) * T(k+1,j) & \text{If operator[k] is '\&'} \\ F(i,k) * F(k+1,j) & \text{If operator[k] is '|'} \\ T(i,k) * T(k+1,j) + F(i,k) * F(k+1) & \text{If operator[k] is '^'} \end{cases}$$

$$\mathrm{Total(i, j) = T(i, j) + F(i, j)}$$

- Ref: https://www.algorithmsandme.com/boolean-parenthesization-problem/